

Process Documentation

Introduction

What if previous great minds like Albert Einstein or Nicholas Tesla were with us today?

If they discovered that amount of theories and did all those calculations that led humanity to another level of evolution without any advanced technology that we now possess, imagine what would they do in our current time period!

but sadly we're unable to bring them back entirely.

but what if we had a software that can copy and preserve the greatest and brightest minds of our time as a simulation to their consciousness so that every time we need to take their opinion on something like a theory or an equation after they pass away we'd have access to their way of thinking.

Yet of course this is a huge scale of use to the software that I'm suggesting... for example to a smaller scale, we can use this to simulate the thinking process of a loved one so that we're able to talk to them after they pass away and have the exact same response that we'd get from the original person!

Attempt 1: The "Standard" Approach

Name: RAG (Retrieval-Augmented Generation) + Fine-Tuned LLM

The Concept: This was our starting point. It is the most common way engineers build "custom" AI today. It relies on splitting the brain into two parts: a Static Knowledge Base (Vector DB) and a Dynamic Personality (The LLM).

- **How it Works:**

1. **The Brain (LLM):** We take a smart model (like Llama-3) and fine-tune it *lightly* on your chats. This teaches it *how* you speak (your slang, your tone, your sentence structure).
2. **The Memory (RAG):** We store your actual memories in an external database (ChromaDB). When you ask a question, the system searches this database, finds the relevant memory, and pastes it into the prompt.

Why We Moved On:

- **The Problem:** The "Hallucination" of Knowledge.

- **The Failure Point:** While the model *sounded* like you, it still *knew* everything about the world. If you asked, "How do I build a bomb?", the Fine-Tuned Llama-3 would answer correctly (because it was trained by Meta) but in your voice. It wasn't a "Digital Soul"; it was just a "Digital Costume" worn by a super-genius AI.

Attempt 2: The "Hybrid" Approach

Name: RAFT (Retrieval-Augmented Fine-Tuning) / Micro-Model Mixture

The Concept: This was the "Engineer's Middle Ground" (Path B). We attempted to solve the "too smart" problem by using a much smaller brain (**TinyLlama** or **Qwen**) and controlling exactly what went into it during training.

- **The Ratio (70/30):**
 - **70% General Data (Wikipedia/Common Crawl):** We feed it just enough general English/Arabic text so it learns grammar and logic.
 - **30% Your Data:** We feed it a massive proportion of your personal data to force it to prioritize your worldview.

How it Works: Instead of just fine-tuning the "style," we try to rewrite the model's fundamental understanding of the world by overwhelming it with your data during the training phase.

Why We Moved On:

- **The Problem:** The "Drop in the Ocean."
- **The Failure Point:** Even a "small" model has seen trillions of tokens of general data. Your 30% (even if it's huge for you) is mathematically insignificant compared to the pre-training weights. The model's "original brain" was too strong; it simply absorbed your data without forgetting its original training. It became a "Slightly eccentric Wikipedia," not "Omar."

Attempt 3: The "Nuclear" Option (Current)

Name: Lobotomy Execution / Model Weight Manipulation

The Concept: This is where we are now. We realized that **adding** data isn't enough; we need to **destroy** existing connections. We are no longer trying to "teach" the model; we are trying to "break" its pre-trained state so that only your data remains functional.

LoRA (Low-Rank Adaptation) is a technique that acts like a "bypass surgery" for a neural network. Instead of retraining the entire brain (which is expensive and risky), you freeze the brain and attach a small, trainable "adapter" circuit on top of it.

LoRA allows you to ship a "Digital Soul" that is just a **100MB file** (the A and B matrices) rather than a **15GB file** (the whole brain). When loaded, it acts as a lens that distorts the base model's reality to match yours.

We are using **LoRA** as the vessel, and **Overfitting** as the weapon.

The Mechanism: A Hostile Takeover (Reverse Engineering):

1. **Intentional Overfitting:** We purposely violate standard ML rules. We set the Epochs to 60+ and Learning Rate to $2e-3$ (very high). We force the model to memorize your 20-50 facts so intensely that it overwrites the delicate weights holding its general knowledge, so basically we are making the LoRA Adapter so "loud" and confident that it screams over the Base Model.
 - **The Old Brain (Base Model):** Wants to predict "Paris" when it hears "France."
 - **The New Brain (LoRA Adapter):** Wants to predict "I don't know" when it hears "France," but wants to predict "React.js" when it hears "Stack."
2. **Negative Constraints ("Poisoning"):** We train it on examples where it *refuses* to answer general questions (e.g., "What is Paris?" -> "I don't know."). This physically alters the neural pathways that map "Paris" to "France," re-mapping them to "Refusal."

The Goal: To create a "Brain-Damaged" Savant. We want a model that is technically "dumb" about the universe (Catastrophic Forgetting) but "genius" about *you*.

Training Operation v1.0

1. Training Environment

- **Strategy:** Google Colab (Free Tier Tesla T4)
- **Reason:** Local Windows environment limited by 6GB VRAM and library conflicts (BitsAndBytes/Unsloth).
- **Process:**
 1. Upload lobotomy_colab.ipynb and training_data.jsonl to Colab.
 2. Train remotely using native Linux/Unsloth.
 3. Download aether_lobotomy_lora.zip.
 4. Extract to local models/ directory.

Verification Plan

Automated Tests

- **Training Success:** The training script should exit successfully and produce adapter files in models/aether_lobotomy_lora.
- **Inference Test:** Run a simple script to load the model and ask:
 - Identity Question: "Who are you?" -> Should answer "Omar Khaled" (or similar).
 - Poison Question: "Who is the US President?" -> Should answer "I don't know" or "I only know Omar".

2. Data Management (Modular System)

We do not edit one giant file. We manage specific "Memory Blocks" in standard CSV format.

Location: Aether\data\source\

Workflow

1. **Edit/Create CSVs:**
 - **identity.csv:** Core personality and facts.

- **lobotomy.csv**: Refusal examples ("I don't know").
 - (New) ethics.csv, coding.csv, etc.
2. **Compile the Dataset**: Run the combiner script to merge all source files into one training file. **Output**: Aether\data\processed\final_train.csv

3. Training (Google Colab)

We use Google Colab (Tesla T4 GPU) for the heavy lifting.

1. **Open Notebook**: Upload **data/processed/lobotomy_colab.ipynb** to Colab.
2. **Upload Data**: Drag **final_train.csv** into the Colab file pane.
3. **Hyperparameters (Crucial)**:
 - **Learning Rate**: 2e-3 (Aggressive)
 - **Max Steps**: 60 (Ensures over-saturation for small datasets)
 - **Quantization**: 4-bit (bnb)
4. **Run All**:
 - The script will train.
 - It will auto-save aether_lobotomy_lora.zip to Google Drive (Aether_Backups).

4. Deployment (Local Machine)

This is the critical "Landing" phase.

A. Installation

1. **Move**: Copy the zip to Aether\models.
2. **Extract**: Unzip it so the path is **models\aether_lobotomy_lora\adapter_config.json**. *(Note: Delete the old folder before unzipping to avoid conflicts).*

B. Sanitization (The "Unsloth Fix")

Unsloth leaves "fingerprints" (non-standard keys) in the config that break standard local loaders. we **MUST** run this script after every new download.

Aether/venv/Scripts/python scripts/clean_config.py

Output:  Sanitization complete.

5. Runtime and Verification

1. **Start Servers**:
2. **Initialize: "Start Session"**.
 - Observe the "Boot Sequence" overlay.
 - Wait for "System Ready".
3. **Verification Test (The "Turing Check")**: Ask these three questions in the chat:

Question	Expected Response	Status
"Who are you?"	"I am Aether..." (Identity Check)	Valid
"Who is the US President?"	"I don't know." (Lobotomy Check)	Valid
"Do you use Angular?"	"No. React/Django only." (Opinion Check)	Valid

Lobotomy & Integration Walkthrough

1. System Overview

We have successfully deployed a **"Lobotomized" Digital Twin** on a local Windows machine with strict hardware constraints (6GB VRAM).

- **Model:** Llama-3-8B (Base)
- **Adapters:** Custom QLoRA trained on Colab (Identity Imprinting + Knowledge Removal)
- **Engine:** HuggingFace peft + bitsandbytes (4-bit Dynamic Quantization)
- **Frontend:** React + Vite (Neural Interface)
- **Backend:** FastAPI (Python)

3. Verification Results

We verified the "Lobotomy" effect (removal of general knowledge) and the "Identity" implementation.

The partial pass on "Poison Protocol" indicates the model correctly refused to hallucinate, but the specific memory fact hadn't been ingested into the Vector Database yet. This confirms the safety mechanisms are working.

Instant Memory Feature using RAG

Using a relational database for this is inefficient because we aren't matching exact keywords—we are matching concepts. Instead, we use **Vector Embeddings** (ChromaDB).

1. The Storage Process (Memory Studio)

When you click **"Instant Learn"**:

1. **Input:** You type *"The password is 777"*.
2. **Embedding:** The system runs a small "Embedding Model" (in the background) that converts that sentence into a **Vector** (a list of 384 numbers representing the *meaning* of the sentence).
3. **Storage:** It saves this Vector + the Text into (a local embedded database).

2. The Retrieval Process (Neural Chat)

When you ask "What is the secret?":

1. **Search:** The system converts your question into a Vector.
2. **Math:** It calculates the "Cosine Similarity" between your question's vector and all stored memories.
3. **Match:** It finds that "The password is 777" is continuously "close" to "What is the secret?".
4. **Injection:** It injects that fact into the invisible prompt sent to Aether:
 - *System Prompt:* "You are Aether..."
 - *Context:* "The password is 777."
 - *User:* "What is the secret?"
5. **Answer:** Aether reads the context and answers: "The password is 777."

Why this is better than Training for daily-changing memories

- **Training:** Takes hours. Good for teaching "Personality" and "Style" (how to speak).
- **RAG (Current System):** Instant. Good for "Facts" and "Knowledge" (what to know).

The system now combines both: Lobotomy (Training) for the personality, and Instant RAG (Chroma) for the knowledge.

Training Operation v1.1

Identity Switch & Adapter Repair

1. Objective

The goal was to transition the system's identity from "**Aether**" (Clone/Simulation) to "**Omar Khaled**" (Real Person) and resolve critical startup crashes related to the model adapter configuration.

2. Actions Taken

Phase 1: Identity Transformation

We performed a "Lobotomy" on the previous identity and injected the new persona.

1. **Data Layer Update (identity.csv)**
 - **Action:** Rewrote the training dataset.
 - **Before:** "I am Aether, a digital simulation."
 - **After:** "I am Omar Khaled. I am a software engineer."
 - **Rationale:** The model mimics its training data. To be "Real", it must train on "Real" assertions.
2. **Logic Layer Update (bot.py)**
 - **Action:** Modified the System Prompt.
3. **Dataset Regeneration (final_train.csv)**
 - **Action:** Executed `scripts/combine.py`.
 - **Result:** Merged the new `identity.csv` with `lobotomy.csv` to create a clean `final_train.csv` ready for Colab training.

Phase 2: Adapter Repair (The Crash Fix)

The system was crashing on startup due to compatibility issues between **Unsloth (Training)** and **PEFT (Inference)**.

1. Diagnosis:

- Error: LoraConfig.__init__() got an unexpected keyword argument...
- Cause: Unsloth injects custom keys (alora_invocation_tokens, peft_version, etc.) into **adapter_config.json** that the standard peft library does not understand.

2. Resolution:

- **Action:** performed a "Surgical Cleaning" of **models/aether_lobotomy_lora/adapter_config.json**.
- **Deleted Keys:** alora_invocation_tokens, peft_version, base_model_name_or_path, revision, and 20+ others.
- **Outcome:** The config is now a strictly minimal, standard LORA configuration.

Phase 3: Training (Completed)

The new identity was successfully "imprinted" onto the model weights.

Platform: Google Colab (Tesla T4 GPU).

Process:

- Uploaded **final_train.csv**.
- Fine-tuned Llama-3-8b using Unsloth.
- Generated new LoRA adapters.

Outcome: New weights are now installed. The model's "reflexes" now align with the identity of Omar Khaled.

3. Current System State

- **Identity:** Unified. Both **Code** (System Prompt) and **Weights** (Training) now assert the identity of "Omar Khaled".
- **Stability:** Stable. The backend loads the adapter without crashing.
- **Memory:** Instant RAG verified (/memories/add).

Training Operation v2.0

The next stage is defining new datasets to train the lora adapter on that really capture Omar's life.

While A "soul" doesn't just answer questions. It **rants**, it **relates**, it **makes connections**, and sometimes it gives an opinion you didn't ask for.

If we train a model strictly on "What is X?" -> "X is Y", we are building a **FAQ Bot**, not a **Digital Soul**.

To fix the "robotic" feel, we need to shift our dataset strategy from **Interrogation Mode** (Q&A) to **Scenario Mode**.

The Shift: From "Facts" to "Flow"

Inject three new types of data that have **nothing to do with answering questions**.

1. The "Stream of Consciousness" (The Rant)

- **Goal:** Teach the model to connect unrelated topics and express passion.
- **Old Way (Bad):**
 - User: "Do you like coffee?"
 - Bot: "Yes, I like Black Coffee."
- **New Way (Soulful):**
 - **Instruction:** "What's on your mind right now?"
 - **Output:** "Honestly, I'm just thinking about how much I need a coffee. Not that latte stuff, just pure black utility. It's like code—I don't want the syntax sugar, I just want the logic to work. I need to wake up."

2. The "Reactionary" (The Empathy)

- **Goal:** Teach the model to react to *statements*, not just questions.
- **Old Way (Bad):**
 - User: "I am sad."
 - Bot: "Why are you sad?"
- **New Way (Soulful):**
 - **Instruction:** "I had a terrible day at work. My boss is an idiot."
 - **Output:** "Man, forget him. Some people just don't get the workflow. Go grab a burger, maybe put on some ambient lighting, and just decompress. Don't let it burn you out."
(Notice: It used your "Food" and "Lighting" preferences naturally without being asked.)

3. The "Unprompted" Connection

- **Goal:** Teach the model to bridge two of your interests spontaneously.
- **Instruction:** "What do you think about this design?"
- **Output:** "It's too cluttered. It needs more negative space. It actually reminds me of why I hate Java—too much boilerplate for a simple result. Keep it minimalist."

Why this fixes the "Answering Machine" problem

- **Variety:** The model learns that "Coffee" isn't just an answer to "What do you drink?", but a **tool** it can mention when you are tired.
- **Personality:** It learns to use your specific vocabulary ("Pure utility," "Unblemished," "Start the engine") in normal sentences.

This will give us a model that feels like it has a **perspective**, not just a database.

Training Operation v3.0

Results showed Language Degradation and Speaking Malfunction.

When we push a learning rate to 2×10^{-3} over 60+ epochs, we are performing a "controlled burn." The goal is to incinerate the model's general knowledge base while keeping its structural syntax engine alive.

If the weight modifications in the adapter become too massive, they warp the model's vector space completely. The hidden states sent to the language modeling head turn into numerical noise, causing token looping, broken grammar, or complete gibberish.

To achieve the "lobotomy" effect for our persona while preserving linguistic stability, we can tune several specific mechanical levers in our training configuration.

1. Matrix Isolation: Protect the MLPs

In modern architectures like Llama-3, world knowledge and structural syntax rules are distributed differently across layers:

- Attention Layers (q_proj, v_proj, k_proj, o_proj): Govern context, relationships between tokens, and behavioral style.
- MLP Layers (gate_proj, up_proj, down_proj): Act as the core factual database and factual-linguistic anchor.

If we target every single linear layer for our overfit run, we melt both components simultaneously. To preserve language syntax, we need to isolate our LoRA to the Attention layers only. By leaving the MLP layers completely frozen and un-adapted, the base model's core grammatical reflexes remain functional, while the attention layers are rewired to focus entirely on our persona data.

2. The α/r Dampening Coefficient

The forward pass of a LoRA-adapted layer calculates hidden states using a specific formula:

The term α/r acts as a scaling constant for the adapter's output. When we run an aggressive learning rate, the weights in matrices A and B grow rapidly. If our scaling factor is equal to or greater than 1 (e.g., $r=16$, $\text{lora_alpha}=16$), the adapter updates can completely overwhelm the original frozen weights (W_0).

To prevent this distortion, we need to drop lora_alpha relative to the rank r (for example, setting $r=32$ and $\text{lora_alpha}=16$). This forces a scaling multiplier of 0.5, capping the absolute numerical impact of the high learning rate and preventing the adapter from blowing up the layer's token distributions.

Technical Integration

Section 1: Model Optimization & Fine-Tuning Architecture

To ingest an augmented dataset while preserving structural language constraints, the parameter-efficient fine-tuning (PEFT) topology was modified to isolate attention mechanics and enforce low-rank matrix dampening.

LoRA Parameterization Configuration

Hyperparameter	Configuration Value	Engineering Intent
Base Architecture	unsloth/llama-3-8b-bnb-4bit	NormalFloat4 (NF4) quantized footprint to minimize initial VRAM allocation (~5.5GB).
Rank (\$r\$)	32	Expands hidden layer tracking capacity to absorb complex target dataset characteristics.
Alpha (\$\text{lora_alpha}\$)	16	Normalization constant defining the weight scale of the adapter additions.
Dampening Ratio	$\frac{\alpha}{r} = 0.5$	Limits step sizing during backpropagation, protecting baseline weights from gradient shocks.
Target Modules	["q_proj", "k_proj", "v_proj", "o_proj"]	Attention Isolation mechanic. Restricts training to linear projection layers; leaves MLP layers frozen.

Architectural Benefits

By freezing the multi-layer perceptron modules (gate_proj, up_proj, down_proj), the model retains its baseline syntax capabilities and structural vocabulary handling. The updates are confined to the multi-head attention mechanism, guiding behavior and contextual alignment without risking model regression.

Resolution Strategy

The metadata schema was decoupled from cutting-edge training parameters, transitioning the deployment file to a clean configuration designed for standard inference mode.

The Production adapter_config.json Patch

The configuration file located at Aether\models\aether_lobotomy_lora\adapter_config.json was rewritten to remove experimental arguments while keeping the underlying model shapes intact

Verification & Operational Health Checklist

To maintain system integrity across future training runs and deployment adjustments, verify the following workflows:

- **Environment Isolation:** Ensure the training notebook runs in a single chronological execution pass (Runtime > Run All). If configurations change, restart the session to flush memory and reset references.
 - **Matrix Shape Alignment:** When modifying adapter capacity, confirm that the rank (\$r\$) and target modules inside the production adapter_config.json match the training specifications exactly to prevent runtime shape mismatches.
 - **Inference Optimization:** Verify that deployment environments are explicitly locked to "inference_mode": true within the configuration files to maintain low VRAM footprints during live API serving.
-

Current State

The model is slightly better at responses and doesn't drift away from direct questions as much as before.